

Technical Design Document

Nimble RX

Nimble @ WashU Robotics
washurobotics@gmail.com

September 12, 2026

Contents

1	Overview	3
2	Problem	4
2.1	The Core Problem	4
2.2	The Core Goal	4
3	Requirements	5
3.1	Out of Scope	5
3.2	Success Criteria	5
4	Architecture	6
4.1	End-to-End Pipeline	6
4.2	Natural Language Instruction Parsing	6
4.3	Item Identification via DataMatrix Barcodes	6
4.4	Per-Item Pick-and-Pace with Safety Verification	7
4.5	Data Collection and Training	7
4.6	Safety Model	7
4.7	High-Level Overview (HLD)	8
4.8	API Design	8
4.9	Data Storage and Model	8
4.10	Application / Component Level Design (LLD)	8
5	FAQs	9
A	Appendix A: Subtitle (Optional)	10
B	Glossary	11
C	References	12

1 Overview

The purpose of this document is to present the technical design of **Nimble RX**, a natural-language-initiated prescription handling system built on Vision-Language-Action (VLA) models. The system is designed to reduce human error and speed up the manual, repetitive tasks relating to prescription handling, by allowing a pharmacist to issue spoken instructions (e.g., “Pick up Box A and Bottle B.”) and have a robotic arm autonomously pick, place, and verify the correct item(s).

This design document is intended for a technical audience with a background in computer science, robotics, or machine learning. Non-technical readers may find high-level summaries useful but should refer to supplementary documentation at the end of this document for background on the underlying mechanisms employed.

2 Problem

Prescription handling today is largely **manual**: pharmacy staff count and verify orders by hand. This is a repetitive task that is prone to human error, and mistakes in this domain carry real consequences.

2.1 The Core Problem

Manual prescription sorting and organization requires a human to:

- Visually identify each individual prescription item.
- Determine, from memory or a written list, which container the prescription item can be found in.
- Physically pick up and place each item.

An autonomous system that generalizes prescription handling would:

- Scale efficiently: handle arbitrary natural-language sorting instructions without task-specific reprogramming.
- Reduce risk: apply automated, repeatable safety checks at every pick and place step.
- Increase trust: confirm instructions back to the user and verify final placement correctness before considering a task complete.

2.2 The Core Goal

The goal is to develop a prescription handling system that can:

- Accept arbitrary natural-language voice instructions (e.g., “Pick up Box A.”).
- Dynamically parse those instructions into a schedule of pick-and-place actions that a model could understand.
- Execute each pick-and-place action with layered safety verification (collision checks and a post-placement correctness check), withholding any action that fails a check rather than executing it blindly.

3 Requirements

Use Cases

- As a pharmacist, I want to give a single spoken instruction to fetch multiple prescriptions in one trip so that I do not need to manually handle each one.
- As a pharmacist, I want the system to read back its understanding of my instruction before acting, so I can catch a misunderstanding before anything is fetched.
- As a pharmacist, I want the robot to withhold an action it is not confident is safe or correct, rather than execute it and risk fetching an invalid prescription.

3.1 Out of Scope

- Multi-arm coordination or handling multiple robots simultaneously.
- Handling of liquid medications or non-rigid/irregular items.

3.2 Success Criteria

Metrics

- Per-item success rate: $\geq 80\%$, measured over 5 attempts per item type.
- Safety effectiveness: $\geq 80\%$ of unsafe or incorrect predicted actions correctly withheld (via collision checks and the CV correctness check).
- *TODO*: define end-to-end latency target from spoken instruction to completed, verified placement.

4 Architecture

Tech Stack

- **VLA:** NORA 1.5
- **VLA Data Collection:** LeRobot (LeRobotDataset)
- **Arm:** WidowX AI Base (7-DOF)
- **Cameras:** Intel RealSense D435i, Logitech (?)
- **CV:** YOLO, OpenCV, Data Matrix (pylibdmtx)
- **Human Interaction:** Whisper (STT), Kokoro (TTS)
- **LLM:** Qwen 3.5 2B
- **Compute:** RoboServer GPU inference (local), WashU Compute Cluster (training)
- **Programming Languages:** Python, JSON

4.1 End-to-End Pipeline

```
"Put the red prescription in Container A."
-> Whisper STT -> schedule parser (local Qwen LLM)
-> [(rx_i, container=i), ...]
-> Kokoro TTS read-back confirmation
-> pre-pick DataMatrix scan pass over workspace
    -> pylibdmtx decode -> {item_id: location} map
-> for each (rx, container):
    -> resolve rx -> item_id -> location via scan map
    -> VLA Inf. #1: pick -> collision check: pass -> continue | fail -> withhold
    -> VLA Inf. #2: place -> collision check: pass -> continue | fail -> withhold
    -> post-place DataMatrix decode: match expected item_id?
        -> pass -> continue | fail/unreadable -> withhold
    -> YOLO CV model: correctness check -> pass -> done | fail -> withhold
```

4.2 Natural Language Instruction Parsing

A spoken instruction is first transcribed by Whisper STT. The resulting text is passed to a local Qwen LLM, which acts as a schedule parser: it converts free-form instructions into a structured list of `(rx_i, container_i)` pairs, one per prescription item referenced. Before execution, Kokoro TTS reads this parsed schedule back to the user, giving them a chance to catch and correct any misparse before any physical action is taken.

4.3 Item Identification via DataMatrix Barcodes

Each prescription bottle/box is labeled with a printed DataMatrix code encoding a short item ID (e.g., `RX003`), which maps to item metadata (name, expected container) in a local lookup table. DataMatrix is used instead of a 1D barcode because it packs more data into a smaller footprint and decodes reliably even when the label is rotated or viewed at an angle, and instead of OCR because decoding a printed code is a far more reliable and unambiguous read than parsing printed text under variable lighting and label orientation.

Identification is performed in two passes using `pylibdmtx` against RGB frames from the environment camera:

- **Pre-pick scan.** Before any VLA inference runs, a frame of the workspace is scanned and every visible DataMatrix code is decoded, producing an `{item_id:`

`location`} map. The schedule parser’s (`rx.i`, `container.i`) pairs are resolved against this map so the VLA is handed a located, identified target rather than an ambiguous description.

- **Post-place verification.** After the place action, the region YOLO identifies as the placed item is cropped and re-decoded. If the decoded ID does not match the expected `rx.i` for that container, the placement is treated as failed and withheld/flagged, the same as a failed collision or CV check.

A failed or unreadable decode (blurry frame, glare, item edge-on to the camera) is always treated as a withhold-and-retry condition rather than a guess.

4.4 Per-Item Pick-and-Place with Safety Verification

For each (`rx`, `container`) pair in the parsed schedule, the system runs two VLA inference passes plus CV/barcode verification:

1. **Pick.** VLA inference #1 produces a pick action for the item, using the location resolved from the pre-pick DataMatrix scan. A collision check evaluates the action before execution; on failure, the action is withheld rather than executed.
2. **Place.** VLA inference #2 produces a place action targeting the specified container. Again, a collision check gates execution.
3. **Verify.** A post-place DataMatrix decode confirms item identity, and a YOLO CV model checks whether the item was placed in the correct container. A pass on both marks the item complete; a fail on either causes the placement to be withheld/flagged rather than silently accepted.

This staged gating (pick $\rightarrow$ place $\rightarrow$ verify) is the system’s core safety mechanism: at no point is an action executed and assumed correct without a corresponding check. Barcode verification adds a hard, unambiguous identity check alongside YOLO’s spatial/placement judgment, so the two checks catch different failure modes (wrong item vs. wrong location).

4.5 Data Collection and Training

- **Data collection.** Data is collected as `LeRobotDataset` episodes, where each episode captures an instruction, a camera frame, and the corresponding 6-DOF grasp/place action chunk. The initial dataset comprises 10 episodes per item type, scaling toward ≥ 50 episodes per item type. Isaac Lab may be used to accelerate collection (e.g., via simulation).
- **Training.** The VLA is trained via supervised fine-tuning (SFT) on NORA 1.5 using its official fine-tuning script. Training and evaluation are performed per-item, which keeps the success signal dense relative to training on full multi-item sequences.
- *Stretch goal:* Direct Preference Optimization (DPO) post-training, pairing successful vs. failed rollouts of the same instruction, using the YOLO CV and DataMatrix verdicts as preference labels.

4.6 Safety Model

The YOLO CV model and the DataMatrix verification step serve complementary roles: YOLO provides spatial localization (where an item/container is, and whether placement geometry looks correct), while DataMatrix decoding provides a hard identity check (is this actually the correct item). Together they act as a post-placement evaluator and

safety outcome predictor, whose combined verdicts can be used to withhold unsafe or incorrect actions and, in the DPO stretch goal, to generate preference labels for further training.

4.7 High-Level Overview (HLD)

List all logical system components:

- Voice input capture and transcription (Whisper STT)
- Instruction-to-schedule parser (local Qwen LLM)
- Read-back confirmation (Kokoro TTS)
- DataMatrix pre-pick scan and post-place verification (pylibdmtx)
- VLA pick/place inference engine (fine-tuned NORA 1.5)
- Collision-checking safety gate (pre-execution)
- Post-placement correctness verification (YOLO CV model)

Include diagrams as needed:

4.8 API Design

List all APIs used for interaction between users or services. Include HTTP methods, payloads, versions, and example requests/responses. Discuss future evolution of the APIs.

4.9 Data Storage and Model

Describe the data model and database choice. Estimate data volume, forecast growth, and justify scalability. Discuss data pipelines, ingestion, and pre-processing if applicable. The item ID $\rightarrow$ metadata lookup table (name, expected container) used to resolve decoded DataMatrix codes should be included in this data model.

4.10 Application / Component Level Design (LLD)

Provide detailed design of each system component, including data flow and control flow diagrams.

USE DRAW.IO for this (can install on your computer)

5 FAQs

question? answer here

A Appendix A: Subtitle (Optional)

Include detailed technical analysis or supporting data.

B Glossary

ex1 example definition

DataMatrix A 2D matrix barcode format capable of encoding data in a small footprint, decodable even when rotated or viewed at an angle.

pylibdmtx Python wrapper for libdmtx, used to decode DataMatrix codes from image frames.

C References

- Service 1: <https://example.com>